

Optimizing compiler

An **optimizing compiler** is a compiler designed to generate code that is optimized in aspects such as minimizing program execution time, memory usage, storage size, and power consumption.^[1] Optimization is generally implemented as a sequence of **optimizing transformations**, a.k.a. **compiler optimizations** – algorithms that transform code to produce semantically equivalent code optimized for some aspect.

Optimization is limited by a number of factors. Theoretical analysis indicates that some optimization problems are NP-complete, or even undecidable.^[2] Also, producing perfectly *optimal* code is not possible since optimizing for one aspect often degrades performance for another (see: superoptimization). Optimization is a collection of heuristic methods for improving resource usage in typical programs.^{[3]:585}

Categorization

Local vs. global scope

Scope describes how much of the input code is considered to apply optimizations.

Local scope optimizations use information local to a basic block.^[4] Since basic blocks contain no control flow statements, these optimizations require minimal analysis, reducing time and storage requirements. However, no information is retained across jumps.

Global scope optimizations, also known as intra-procedural optimizations, operate on individual functions.^[4] This gives them more information to work with but often makes expensive computations necessary. Worst-case assumptions need to be made when function calls occur or global variables are accessed because little information about them is available.

Peephole optimization

Peephole optimizations are usually performed late in the compilation process after machine code has been generated. This optimization examines a few adjacent instructions (similar to "looking through a peephole" at the code) to see whether they can be replaced by a single instruction or a shorter sequence of instructions.^{[3]:554} For instance, a multiplication of a value by two might be more efficiently executed by left-shifting the value or by adding the value to itself (this example is also an instance of strength reduction).

Inter-procedural optimization

Interprocedural optimizations analyze all of a program's source code. The more information available, the more effective the optimizations can be. The information can be used for various optimizations, including function inlining, where a call to a function is replaced by a copy of the function body.

Link-time optimization

Link-time optimization (LTO), or whole-program optimization, is a more general class of interprocedural optimization. During LTO, the compiler has visibility across translation units which allows it to perform more aggressive optimizations like cross-module inlining and devirtualization.

Machine and object code optimization

Machine code optimization involves using an object code optimizer to analyze the program after all machine code has been linked. Techniques such as macro compression, which conserves space by condensing common instruction sequences, become more effective when the entire executable task image is available for analysis.^[5]

Language-independent vs. language-dependent

Most high-level programming languages share common programming constructs and abstractions, such as branching constructs (if, switch), looping constructs (for, while), and encapsulation constructs (structures, objects). Thus, similar optimization techniques can be used across languages. However, certain language features make some optimizations difficult. For instance, pointers in C and C++ make array optimization difficult; see alias analysis. However, languages such as PL/I that also support pointers implement optimizations for arrays. Conversely, some language features make certain optimizations easier. For example, in some languages, functions are not permitted to have side effects. Therefore, if a program makes several calls to the same function with the same arguments, the compiler can infer that the function's result only needs to be computed once. In languages where functions are allowed to have side effects, the compiler can restrict such optimization to functions that it can determine have no side effects.

Machine-independent vs. machine-dependent

Many optimizations that operate on abstract programming concepts (loops, objects, structures) are independent of the machine targeted by the compiler, but many of the most effective optimizations are those that best exploit special features of the target platform. Examples are instructions that do several things at once, such as decrement register and branch if not zero.

The following is an instance of a local machine-dependent optimization. To set a register to 0, the obvious way is to use the constant '0' in an instruction that sets a register value to a constant. A less obvious way is to XOR a register with itself or subtract it from itself. It is up to the compiler to know which instruction variant to use. On many RISC machines, both instructions would be equally appropriate, since they would both be the same length and take the same time. On many other microprocessors such as the Intel x86 family, it turns out that the XOR variant is shorter and probably faster, as there will be no need to decode an immediate operand, nor use the internal "immediate operand register"; the same applies on IBM System/360 and successors for the subtract variant.^[6] A potential problem with this is that XOR or subtract may introduce a data dependency on the previous value of the register, causing a pipeline stall, which occurs when the processor must delay execution of an instruction because it depends on the result of a previous instruction. However, processors often treat the XOR of a register with itself or the subtract of a register from itself as a special case that does not cause stalls.

Factors affecting optimization

Target machine

Whether particular optimizations can and should be applied may depend on the characteristics of the target machine. Some compilers such as GCC and Clang parameterize machine-dependent factors so that they can be used to optimize for different machines.^[7]

Target CPU architecture

- Number of registers: Registers can be used to optimize for performance. Local variables can be stored in registers instead of the stack. Temporary/intermediate results can be accessed in registers instead of slower memory.
- RISC vs. CISC: CISC instruction sets often have variable instruction lengths,^[8] often have a larger number of possible instructions that can be used, and each instruction could take differing amounts of time. RISC instruction sets attempt to limit the variability in each of these: instruction sets are usually constant in length, with few exceptions, there are usually fewer combinations of registers and memory operations, and the instruction issue rate (the number of instructions completed per time period, usually an integer multiple of the clock cycle) is usually constant in cases where memory latency is not a factor. There may be several ways of carrying out a certain task, with CISC usually offering more alternatives than RISC. Compilers have to know the relative costs among the various instructions and choose the best instruction sequence (see instruction selection).
- Pipelines: A pipeline is a CPU broken up into an assembly line. It allows the use of parts of the CPU for different instructions by breaking up the execution of instructions into various stages: instruction decode, address decode, memory fetch, register fetch, compute, register store, etc. One instruction could be in the register store stage, while another could be in the register fetch stage. Pipeline conflicts occur when an instruction in one stage of the pipeline depends on the result of another instruction ahead of it in the pipeline but not yet completed. Pipeline conflicts can lead to pipeline stalls: where the CPU wastes cycles waiting for a conflict to resolve. Compilers can *schedule*, or reorder, instructions so that pipeline stalls occur less frequently.
- Number of functional units: Some CPUs have several ALUs and FPUs that allow them to execute multiple instructions simultaneously. There may be restrictions on which instructions can pair with which other instructions ("pairing" is the simultaneous execution of two or more instructions), and which functional unit can execute which instruction. They also have issues similar to pipeline conflicts. Instructions can be scheduled so that the functional units are fully loaded.

Machine architecture

- CPU cache size and type (direct mapped, 2-/4-/8-/16-way associative, fully associative): Techniques such as inline expansion and loop unrolling may increase the size of the generated code and reduce code locality. The program may slow down drastically if a highly used section of code (like inner loops in various algorithms) no longer fits in the cache as a result of optimizations that increase code size. Also, caches that are not fully associative have higher chances of cache collisions even in an unfilled cache.
- Cache/memory transfer rates: These give the compiler an indication of the penalty for cache misses. This is used mainly in specialized applications.

Intended use

- **Debugging:** During development, optimizations are often disabled to speed compilation or to make the executable code easier to debug. Optimizing transformations, particularly those that reorder code, can make it difficult to relate the executable code to the source code.
- **General-purpose use:** Prepackaged software is often expected to run on a variety of machines that may share the same instruction set but have different performance characteristics. The code may not be optimized to any particular machine or may be tuned to work best on the most popular machine while working less optimally on others.
- **Special-purpose use:** If the software is compiled for machines with uniform characteristics, then the compiler can heavily optimize the generated code for those machines.

Notable cases include code designed for parallel and vector processors, for which special parallelizing compilers are used.

Firmware for an embedded system can be optimized for the target CPU and memory. System cost or reliability may be more important than the code speed. For example, compilers for embedded software usually offer options that reduce code size at the expense of speed. The code's timing may need to be predictable, rather than as fast as possible, so code caching might be disabled, along with compiler optimizations that require it.

Common themes

Optimization includes the following, sometimes conflicting themes.

Optimize the common case

The common case may have unique properties that allow a *fast path* at the expense of a *slow path*. If the fast path is taken more often, the result is better overall performance.

Avoid redundancy

Reuse results that are already computed and store them for later use, instead of recomputing them.

Less code

Remove unnecessary computations and intermediate values. Less work for the CPU, cache, and memory usually results in faster execution. Alternatively, in embedded systems, less code brings a lower product cost.

Fewer jumps by using *straight line code*, also called *branch-free code*

Less complicated code. Jumps (conditional or unconditional branches) interfere with the prefetching of instructions, thus slowing down code. Using inlining or loop unrolling can reduce branching, at the cost of increasing binary file size by the length of the repeated code. This tends to merge several basic blocks into one.

Locality

Code and data that are accessed closely together in time should be placed close together in memory to increase spatial locality of reference.

Exploit the memory hierarchy

Accesses to memory are increasingly more expensive for each level of the memory hierarchy, so place the most commonly used items in registers first, then caches, then main memory, before going to disk.

Parallelize

Reorder operations to allow multiple computations to happen in parallel, either at the instruction, memory, or thread level.

More precise information is better

The more precise the information the compiler has, the better it can employ any or all of these optimization techniques.

Runtime metrics can help

Information gathered during a test run can be used in profile-guided optimization. Information gathered at runtime, ideally with minimal overhead, can be used by a JIT compiler to dynamically improve optimization.

Strength reduction

Replace complex, difficult, or expensive operations with simpler ones. For example, replacing division by a constant with multiplication by its reciprocal, or using induction variable analysis to replace multiplication by a loop index with addition.

Specific techniques

Loop optimizations

Loop optimization acts on the statements that make up a loop, such as a *for* loop, for example loop-invariant code motion. Loop optimizations can have a significant impact because many programs spend a large percentage of their time inside loops.^{[3]:596}

Some optimization techniques primarily designed to operate on loops include:

Induction variable analysis

Roughly, if a variable in a loop is a simple linear function of the index variable, such as $j := 4*i + 1$, it can be updated appropriately each time the loop variable is changed. This is a strength reduction and also may allow the index variable's definitions to become dead code.^{[3]:596–598} This information is also useful for bounds-checking elimination and dependence analysis, among other things.

Loop fission or loop distribution

Loop fission attempts to break a loop into multiple loops over the same index range with each new loop taking only a part of the original loop's body. This can improve locality of reference to both the data being accessed within the loop and the code in the loop's body.

Loop fusion or loop combining or loop ramming or loop jamming

Another technique that attempts to reduce loop overhead. When two adjacent loops would iterate the same number of times regardless of whether that number is known at compile time, their bodies can be combined as long as they do not refer to each other's data.

Loop inversion

This technique changes a standard *while* loop into a *do/while* (also known as *repeat/until*) loop wrapped in an *if* conditional, reducing the number of jumps by two, for cases when the loop is executed. Doing so duplicates the condition check (increasing the size of the code) but is more efficient because jumps usually cause a pipeline stall. Additionally, if the initial condition is known at compile-time and is known to be side-effect-free, the *if* guard can be skipped.

Loop interchange

These optimizations exchange inner loops with outer loops. When the loop variables index into an array, such a transformation can improve the locality of reference, depending on the array's layout.

Loop-invariant code motion

If a quantity is computed inside a loop during every iteration, and its value is the same for each iteration, it can vastly improve efficiency to hoist it outside the loop and compute its value just once before the loop begins.^{[3]:596} This is particularly important with the

address-calculation expressions generated by loops over arrays. For correct implementation, this technique must be used with loop inversion, because not all code is safe to be hoisted outside the loop.

Loop nest optimization

Some pervasive algorithms such as matrix multiplication have very poor cache behavior and excessive memory accesses. Loop nest optimization increases the number of cache hits by operating over small blocks and by using a loop interchange.

Loop reversal

Loop reversal reverses the order in which values are assigned to the index variable. This is a subtle optimization that can help eliminate dependencies and thus enable other optimizations. Furthermore, on some architectures, loop reversal contributes to smaller code, as when the loop index is being decremented, the condition that needs to be met for the running program to exit the loop is a comparison with zero. This is often a special, parameter-less instruction, unlike a comparison with a number, which needs the number to compare to. Therefore, the amount of bytes needed to store the parameter is saved by using the loop reversal. Additionally, if the comparison number exceeds the size of word of the platform, in standard loop order, multiple instructions would need to be executed to evaluate the comparison, which is not the case with loop reversal.

Loop unrolling

Unrolling duplicates the body of the loop multiple times, to decrease the number of times the loop condition is tested and the number of jumps; tests and jumps can hurt performance by impairing the instruction pipeline. A "fewer jumps" optimization. Completely unrolling a loop eliminates all overhead, but requires that the number of iterations be known at compile time.

Loop splitting

Loop splitting attempts to simplify a loop or eliminate dependencies by breaking it into multiple loops that have the same bodies but iterate over different contiguous portions of the index range. A useful special case is loop peeling, which can simplify a loop with a problematic first iteration by performing that iteration separately before entering the loop.

Loop unswitching

Unswitching moves a conditional from inside a loop to outside the loop by duplicating the loop's body inside each of the if and else clauses of the conditional.

Software pipelining

The loop is restructured in such a way that work done in an iteration is split into several parts and done over several iterations. In a tight loop, this technique hides the latency between loading and using values.

Automatic parallelization

A loop is converted into multi-threaded or vectorized (or even both) code to use multiple processors simultaneously in a shared-memory multiprocessor (SMP) machine, including multi-core machines.

Prescient store optimizations

Prescient store optimizations allow store operations to occur earlier than would otherwise be permitted in the context of threads and locks. The process needs some way of knowing ahead of time what value will be stored by the assignment that it should have followed. The purpose of this relaxation is to allow compiler optimization to perform certain kinds of code rearrangements that preserve the semantics of properly synchronized programs.^[9]

Data-flow optimizations

Data-flow optimizations, based on data-flow analysis, primarily depend on how certain properties of data are propagated by control edges in the control-flow graph. Some of these include:

Common subexpression elimination

In the expression $(a + b) - (a + b) / 4$, "common subexpression" refers to the duplicated $(a + b)$. Compilers implementing this technique realize that $(a + b)$ will not change, and so only calculate its value once.^{[3]:592–594}

Constant folding and propagation

Replacing expressions consisting of constants (e.g., $3 + 5$) with their final value (8) at compile time, rather than doing the calculation in run-time.^[10] Used in most modern languages.

Induction variable recognition and elimination

See discussion above about *induction variable analysis*.

Alias classification and pointer analysis

In the presence of pointers, it is difficult to make any optimizations at all, since potentially any variable can have been changed when a memory location is assigned to. By specifying which pointers can alias which variables, unrelated pointers can be ignored.

Dead-store elimination

Removal of assignments to variables that are not subsequently read, either because the lifetime of the variable ends or because of a subsequent assignment that will overwrite the first value.

SSA-based optimizations

These optimizations are intended to be done after transforming the program into a special form called static single-assignment form, in which every variable is assigned in only one place. Although some function without SSA, they are most effective with SSA. Many optimizations listed in other sections also benefit with no special changes, such as register allocation.

Global value numbering

GVN eliminates redundancy by constructing a value graph of the program, and then determining which values are computed by equivalent expressions. GVN can identify some redundancy that common subexpression elimination cannot, and vice versa.

Sparse conditional constant propagation

Combines constant propagation, constant folding, and dead-code elimination, and improves upon what is possible by running them separately.^{[11][12]} This optimization symbolically executes the program, simultaneously propagating constant values and eliminating portions of the control-flow graph that this makes unreachable.

Code generator optimizations

Register allocation

The most frequently used variables should be kept in processor registers for the fastest access. To find which variables to put in registers, an interference-graph is created. Each variable is a vertex and when two variables are used at the same time (have an intersecting live range) they have an edge between them. This graph is colored using for

example Chaitin's algorithm using the same number of colors as there are registers. If the coloring fails one variable is "spilled" to memory and the coloring is retried.

Instruction selection

Most architectures, particularly CISC architectures and those with many addressing modes, offer several different ways of performing a particular operation, using entirely different sequences of instructions. The job of the instruction selector is to do a good job overall of choosing which instructions to implement which operators in the low-level intermediate representation with. For example, on many processors in the 68000 family and the x86 architecture, complex addressing modes can be used in statements like `leal 25(a1, d5*4), a0`, allowing a single instruction to perform a significant amount of arithmetic with less storage.

Instruction scheduling

Instruction scheduling is an important optimization for modern pipelined processors, which avoids stalls or bubbles in the pipeline by clustering instructions with no dependencies together, while being careful to preserve the original semantics.

Rematerialization

Rematerialization recalculates a value instead of loading it from memory, eliminating an access to memory. This is performed in tandem with register allocation to avoid spills.

Code factoring

If several sequences of code are identical, or can be parameterized or reordered to be identical, they can be replaced with calls to a shared subroutine. This can often share code for subroutine set-up and sometimes tail-recursion.^[13]

Trampolines

Many CPUs have smaller subroutine call instructions to access low memory. A compiler can save space by using these small calls in the main body of code. Jump instructions in low memory can access the routines at any address. This multiplies space savings from code factoring.^[13]

Reordering computations

Based on integer linear programming, restructuring compilers enhance data locality and expose more parallelism by reordering computations. Space-optimizing compilers may reorder code to lengthen sequences that can be factored into subroutines.

Functional language optimizations

Although many of these also apply to non-functional languages, they either originate in or are particularly critical in functional languages such as Lisp and ML.

Tail-call optimization

A function call consumes stack space and involves some overhead related to parameter passing and flushing the instruction cache. Tail-recursive algorithms can be converted to iteration through a process called tail-recursion elimination or tail-call optimization.

Deforestation (data structure fusion)

In languages where it is common for a sequence of transformations to be applied to a list, deforestation attempts to remove the construction of intermediate data structures.

Partial evaluation

Computations that produce the same output regardless of the dynamic input at runtime can be evaluated at compile time.

Other optimizations

Bounds-checking elimination

Many languages, such as Java, enforce bounds checking of all array accesses. This is a severe performance bottleneck on certain applications such as scientific code. Bounds-checking elimination allows the compiler to safely remove bounds checking in many situations where it can determine that the index must fall within valid bounds; for example, if it is a simple loop variable.

Branch-offset optimization (machine dependent)

Choose the shortest branch displacement that reaches the target.

Code-block reordering

Code-block reordering alters the order of the basic blocks in a program to reduce conditional branches and improve the locality of reference.

Dead-code elimination

Removes instructions that will not affect the behaviour of the program, for example, definitions that have no uses, called dead code. This reduces code size and eliminates unnecessary computation.

Factoring out of invariants (loop invariants)

If an expression is carried out both when a condition is met and is not met, it can be written just once outside of the conditional statement. Similarly, if certain types of expressions (e.g., the assignment of a constant into a variable) appear inside a loop, they can be moved out of it because their effect will be the same no matter if they're executed many times or just once. This is also known as total redundancy elimination. A similar but more powerful optimization is partial-redundancy elimination (PRE).

Inline expansion or macro expansion

When some code invokes a procedure, it is possible to directly insert the body of the procedure inside the calling code rather than transferring control to it. This saves the overhead related to procedure calls, as well as providing an opportunity for many different parameter-specific optimizations, but comes at the cost of space; the procedure body is duplicated each time the procedure is called inline. Generally, inlining is useful in performance-critical code that makes a large number of calls to small procedures. This is a "fewer jumps" optimization. The statements of imperative programming languages are also an example of such an optimization. Although statements could be implemented with function calls they are almost always implemented with code inlining.

Jump threading

In this optimization, consecutive conditional jumps predicated entirely or partially on the same condition are merged.

For example:

```
if (cond) {  
    foo();  
}  
if (cond) {  
    bar();  
}  
  
// becomes:  
if (cond) {  
    foo();  
    bar();  
}
```

and:

```

if (cond) {
    foo();
}
if (!cond) {
    bar();
}

// becomes:
if (cond) {
    foo();
} else {
    bar();
}

```

Macro compression

A space optimization that recognizes common sequences of code, creates subprograms ("code macros") that contain the common code, and replaces the occurrences of the common code sequences with calls to the corresponding subprograms.^[5] This is most effectively done as a machine code optimization, when all the code is present. The technique was first used to conserve space in an interpretive byte stream used in an implementation of Macro Spibol on microcomputers.^[14] The problem of determining an optimal set of macros that minimizes the space required by a given code segment is known to be NP-complete,^[5] but efficient heuristics attain near-optimal results.^[15]

Reduction of cache collisions

(e.g., by disrupting alignment within a page)

Stack-height reduction

Rearrange an expression tree to minimize resources needed for expression evaluation.

Test reordering

If we have two tests that are the condition for something, we can first deal with the simpler tests (e.g., comparing a variable to something) and only then with the complex tests (e.g., those that require a function call). This technique complements lazy evaluation, but can be used only when the tests are not dependent on one another. Short-circuiting semantics can make this difficult.

Interprocedural optimizations

Interprocedural optimization works on the entire program, across procedure and file boundaries. It works tightly with intraprocedural counterparts, carried out with the cooperation of a local part and a global part. Typical interprocedural optimizations are procedure inlining, interprocedural dead-code elimination, interprocedural constant propagation, and procedure reordering. As usual, the compiler needs to perform interprocedural analysis before its actual optimizations. Interprocedural analyses include alias analysis, array access analysis, and the construction of a call graph.

Interprocedural optimization is common in modern commercial compilers from SGI, Intel, Microsoft, and Sun Microsystems. For a long time, the open source GCC was criticized for a lack of powerful interprocedural analysis and optimizations, though this is now improving.^[16] Another open-source compiler with full analysis and optimization infrastructure is Open64.

Due to the extra time and space required by interprocedural analysis, most compilers do not perform it by default. Users must use compiler options explicitly to tell the compiler to enable interprocedural analysis and other expensive optimizations.

Practical considerations

There can be a wide range of optimizations that a compiler can perform, ranging from simple and straightforward optimizations that take little compilation time to elaborate and complex optimizations that involve considerable amounts of compilation time.^{[3]:15} Accordingly, compilers often provide options to their control command or procedure to allow the compiler user to choose how much optimization to request; for instance, the IBM FORTRAN H compiler allowed the user to specify no optimization, optimization at the registers level only, or full optimization.^{[3]:737} By the 2000s, it was common for compilers, such as Clang, to have several compiler command options that could affect a variety of optimization choices, starting with the familiar `-O2` switch.^[17]

An approach to isolating optimization is the use of so-called post-pass optimizers (some commercial versions of which date back to mainframe software of the late 1970s).^[18] These tools take the executable output by an optimizing compiler and optimize it even further. Post-pass optimizers usually work on the assembly language or machine code level (in contrast with compilers that optimize intermediate representations of programs). One such example is the Portable C Compiler (PCC) of the 1980s, which had an optional pass that would perform post-optimizations on the generated assembly code.^{[3]:736}

Another consideration is that optimization algorithms are complicated and, especially when being used to compile large, complex programming languages, can contain bugs that introduce errors in the generated code or cause internal errors during compilation. Compiler errors of any kind can be disconcerting to the user, but especially so in this case, since it may not be clear that the optimization logic is at fault.^[19] In the case of internal errors, the problem can be partially ameliorated by a "fail-safe" programming technique in which the optimization logic in the compiler is coded such that a failure is trapped, a warning message issued, and the rest of the compilation proceeds to successful completion.^[20]

History

Early compilers of the 1960s were often primarily concerned with simply compiling code correctly or efficiently, such that compile times were a major concern. One notable early optimizing compiler was the IBM FORTRAN H compiler of the late 1960s.^{[3]:737} Another of the earliest and important optimizing compilers, that pioneered several advanced techniques, was that for BLISS (1970), which was described in *The Design of an Optimizing Compiler* (1975).^{[3]:740,779} By the late 1980s, optimizing compilers were sufficiently effective that programming in assembly language declined. This co-evolved with the development of RISC chips and advanced processor features such as superscalar processors, out-of-order execution, and speculative execution, which were designed to be targeted by optimizing compilers rather than by human-written assembly code.

List of static code analyses

- Alias analysis
- Pointer analysis
- Shape analysis
- Escape analysis

- Array-access analysis
- Dependence analysis
- Control-flow analysis
- Data-flow analysis
 - Use-define chain analysis
 - Live-variable analysis
 - Available expression analysis

See also

- Algorithmic efficiency
- Compile-time function execution
- Full-employment theorem
- Just-in-time compilation (JIT)
- Kildall's method
- Profile-guided optimization
- Program optimization
- Superoptimization

References

1. Godbolt, Matt (November 12, 2019). "Optimizations in C++ Compilers" (<https://queue.acm.org/detail.cfm?id=3372264>). *ACM Queue*. Vol. 17, no. 5.
2. "Lecture 15: NP-completeness, Optimization and Separation" (<https://karthik.ise.illinois.edu/courses/ie511/lectures-sp-21/lecture-15.pdf>) (PDF). *IE 511: Integer Programming, Spring 2021*.
3. Aho, Alfred V.; Sethi, Ravi; Ullman, Jeffrey D. (1986). *Compilers: Principles, Techniques, and Tools*. Reading, Massachusetts: Addison-Wesley. ISBN 0-201-10088-6.
4. Cooper, Keith D.; Torczon, Linda (2003) [2002-01-01]. *Engineering a Compiler*. Morgan Kaufmann. pp. 404, 407. ISBN 978-1-55860-698-2.
5. Goss, Clinton F. (August 2013) [First published June 1986]. *Machine Code Optimization – Improving Executable Object Code* (https://ClintGoss.com/mco/Goss_1986_MachineCodeOptimization.pdf) (PDF) (Ph.D. dissertation). Vol. Computer Science Department Technical Report #246. Courant Institute, New York University. arXiv:1308.4815 (<https://arxiv.org/abs/1308.4815>). Bibcode:2013arXiv1308.4815G (<https://ui.adsabs.harvard.edu/abs/2013arXiv1308.4815G>). Archived (https://ghostarchive.org/archive/20221009/https://ClintGoss.com/mco/Goss_1986_MachineCodeOptimization.pdf) (PDF) from the original on 2022-10-09. Retrieved 22 Aug 2013.
 - Clinton F. Goss (2013) [1986]. *Machine Code Optimization - Improving Executable Object Code* (<https://ClintGoss.com/mco/>) (PhD thesis). Courant Institute, New York University.
6. *A Programmer's Introduction to IBM System/360 Assembly Language* (https://bitsavers.trailing-edge.com/pdf/ibm/360/training/GC20-1646-5_A_Programmers_Introduction_to_IBM_System360_Assembly_Language_196907.pdf) (PDF). IBM. p. 42. GC20-1645-5.
7. "GCC – Machine-Dependent Options" (<https://gcc.gnu.org/onlinedocs/gcc/gcc-command-options/machine-dependent-options.html>). GNU Project.

8. "RISC vs. CISC" (<https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/>). *cs.stanford.edu*. Retrieved 2024-10-15.
9. James Gosling; Bill Joy; Guy Steele. "17 Threads and Locks" (<http://titanium.cs.berkeley.edu/doc/java-langs-spec-1.0/17.doc.html#45376>). *The Java Language Specification* (<http://titanium.cs.berkeley.edu/doc/java-langs-spec-1.0/index.html>) (1.0 ed.). 17.8 Prescient Store Actions.
10. Muchnick, Steven; Muchnick and Associates (15 August 1997). *Advanced Compiler Design Implementation* (<https://archive.org/details/advancedcompiler00much>). Morgan Kaufmann. pp. 329 (<https://archive.org/details/advancedcompiler00much/page/329>)–. ISBN 978-1-55860-320-2. "constant folding."
11. Wegman, Mark N.; Zadeck, F. Kenneth (April 1991). "Constant Propagation with Conditional Branches" (<https://bears.ece.ucsb.edu/class/ece253/papers/wegman91.pdf>) (PDF). *ACM Transactions on Programming Languages and Systems*. **13** (2): 181–210. doi:10.1145/103135.103136 (<https://doi.org/10.1145%2F103135.103136>).
12. Click, Clifford; Cooper, Keith. (March 1995). "Combining Analyses, Combining Optimizations" (<https://scholarship.rice.edu/bitstream/handle/1911/96451/TR95-252.pdf?sequence=1&isAllowed=y>) (PDF). *ACM Transactions on Programming Languages and Systems*. **17** (2): 181–196. doi:10.1145/201059.201061 (<https://doi.org/10.1145%2F201059.201061>).
13. Cx51 Compiler Manual, version 09.2001, p. 155, Keil Software Incorporated.
14. Dewar, Robert B. K.; Golumbic, Martin Charles; Goss, Clinton F. (August 2013) [October 1979]. *MICRO SPITBOL*. Computer Science Department Technical Report. Vol. 11. Courant Institute of Mathematical Sciences. arXiv:1308.6096 (<https://arxiv.org/abs/1308.6096>). Bibcode:2013arXiv1308.6096D (<https://ui.adsabs.harvard.edu/abs/2013arXiv1308.6096D>).
15. Golumbic, Martin Charles; Dewar, Robert B. K.; Goss, Clinton F. (1980). "Macro Substitutions in MICRO SPITBOL – a Combinatorial Analysis". *Proceedings of the 11th Southeastern Conference on Combinatorics, Graph Theory and Computing, Congressus Numerantium, Utilitas Math., Winnipeg, Canada*. 11th Southeastern Conference on Combinatorics, Graph Theory and Computing. Vol. 29. pp. 485–495.
16. Glazunov, N. M. (November 25, 2012). "Foundations of Scientific Research". arXiv:1212.1651 (<https://arxiv.org/abs/1212.1651>) [cs.OH (<https://arxiv.org/archive/cs/OH>)].
17. Guelton, Serge (August 5, 2019). "Customize the compilation process with Clang: Optimization options" (<https://developers.redhat.com/blog/2019/08/05/customize-the-compilation-process-with-clang-optimization-options/>). Red Hat.
18. Evans, Michael (December 1982). "Software engineering for the Cobol environment" (<https://dl.acm.org/doi/10.1145/358728.358732>). *Communications of the ACM*. **25** (12): 874–882. doi:10.1145/358728.358732 (<https://doi.org/10.1145%2F358728.358732>). Retrieved 2013-08-10.
19. Sun, Chengnian; et al. (July 18–20, 2016). "Toward understanding compiler bugs in GCC and LLVM" (<https://dl.acm.org/citation.cfm?id=2931074&CFID=827491623&CFTOKEN=57038897>). *Proceedings of the 25th International Symposium on Software Testing and Analysis*. Issta 2016. pp. 294–305. doi:10.1145/2931037.2931074 (<https://doi.org/10.1145%2F2931037.2931074>). ISBN 9781450343909. S2CID 8339241 (<https://api.semanticscholar.org/CorpusID:8339241>).
20. Schilling, Jonathan L. (August 1993). "Fail-safe programming in compiler optimization". *ACM SIGPLAN Notices*. **28** (8): 39–42. doi:10.1145/163114.163118 (<https://doi.org/10.1145%2F163114.163118>). S2CID 2224606 (<https://api.semanticscholar.org/CorpusID:2224606>).

External links

- [Optimization manuals \(https://agner.org/optimize/#manuals\)](https://agner.org/optimize/#manuals) by Agner Fog – documentation about x86 processor architecture and low-level code optimization
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Optimizing_compiler&oldid=1339030242"